

进程同步和互斥

信号量

◆ Posix中的有名信号量使用步骤

- (1) 创建/打开有名信号量
- (2) 关闭信号量
- (3) 删除信号量

◆ System V中的信号量集使用步骤

- (1) 信号量集的结构体信息
- (2) 创建/获取信号量集
- (3) 设置信号量集合中具体信号量的值
- (4) 对信号量集的操作
- (5) 获取/删除信号量集合中具体信号量的值

Posix有名信号量- 创建/打开有名信号量

函数原型: `sem_t * sem_open(const char *, int, ...);`
`sem_t *sem_open(const char* name, int oflag, mode_t mode, int value);`

参数: 参数一: 有名信号量的名称, 自定义

参数二: 对信号量的操作权限

`O_CREAT|O_EXCL`: 创建, `O_EXCL`存在创建失败

参数三: 用户操作的权限: `0666`

参数四: 信号量的初值

返回值: 成功信号量指针; 失败返回NULL

Posix信号量-

申请资源、释放资源

功能：申请资源

函数原型：int sem_wait(sem_t *);

参数：信号量的地址

注意：信号量资源减1

功能：释放资源

函数原型：int sem_post(sem_t *);

参数：信号量的地址

注意：信号量资源加1

Posix有名信号量- 关闭、删除信号量

函数原型：int sem_close(sem_t *);

参数：信号量指针

函数原型：int sem_unlink(const char *);

参数：信号量名称，创建时自定义的

示例代码 1 of 2

```
int main(int argc,char**argv)
{
    const char *semName="sem";
    sem_t *sem=sem_open(semName, O_CREAT|O_EXCL,0666,0);
    pid_t pid=fork();
    if(pid>0)
    {
        printf("hello\n");
        sem_post(sem);
    }
}
```

示例代码 2 of 2

```
else if(pid==0)
{
    sem_wait(sem);
    printf("world\n");
    sem_close(sem);
    sem_unlink(semName);
}
else
{
    printf("fork failed\n");
}
return 0;
}
```

System V信号量集函数

所需头文件	<pre>#include <sys/types.h> #include <sys/ipc.h> #include <sys/sem.h></pre>
函数原型	<pre>int semget(key_t key, int nsems, int semflg);</pre>
函数参数	key: 和信号灯集关联的key值
	nsems: 信号灯集中包含的信号灯数目
	semflg: 信号灯集的访问权限, 通常为IPC_CREAT 0666
函数返回值	成功: 信号灯集ID
	出错: -1

System V信号量集函数

所需头文件	<pre>#include <sys/types.h> #include <sys/ipc.h> #include <sys/sem.h></pre>
函数原型	<pre>int semop (int semid, struct sembuf *opsptr, size_t nops);</pre>
函数参数	semid: 信号灯集ID <pre>struct sembuf { short sem_num; // 要操作的信号灯的编号 short sem_op; // 0: 等待, 直到信号灯的值变成0 // 1: 释放资源, V操作 // -1: 分配资源, P操作 short sem_flg; // 0, IPC_NOWAIT, SEM_UNDO };</pre> nops: 要操作的信号灯的个数
函数返回值	成功: 0
	出错: -1

System V信号量集函数

所需头文件	#include <sys/types.h> #include <sys/ipc.h> #include <sys/sem.h>	
函数原型	int semctl (int semid, int semnum, int cmd.../*union semun arg*/);	
函数参数	semid: 信号灯集ID	
	semnum: 要修改的信号灯编号	
	cmd:	GETVAL: 获取信号灯的值 SETVAL: 设置信号灯的值 IPC_RMID: 从系统中删除信号灯集合
函数返回值	成功: 0 如果是GETVAL返回的是信号灯的資源	
	出错: -1	

示例代码 1 of 2: 申请资源

```
#include <stdio.h>
#include <sys/sem.h>
int main()
{
    //1.创建信号量
    key_t key=123;
    int sem_id=semget(key,1,IPC_CREAT|0666);
    //2.对信号量集中具体的信号量进行操作
    semctl(sem_id, 0, SETVAL,10);
    printf("即将申请资源\n");
    //3.对该信号量集进行操作
    struct sembuf buf;
```

示例代码 2 of 2: 申请资源

```
buf.sem_flg=0;
buf.sem_op=-11;
buf.sem_num=0;
semop(sem_id, &buf, 1);
printf("申请资源完成\n");
printf("===执行任务===\n");
//4.获取信号量集中具体信号量的资源值
int value=semctl(sem_id, 0, GETVAL);
printf("value=%d\n",value);
//5.删除信号量集
semctl(sem_id, 0, IPC_RMID);
return 0;
}
```

示例代码 1 of 1: 释放资源

```
int main(int argc, const char * argv[])
{
    key_t key=123;
    int sem_id=semget(key, 1, IPC_CREAT);
    printf("即将释放资源\n");
    struct sembuf buf;
    buf.sem_flg=0;
    buf.sem_op=13;
    buf.sem_num=0;
    semop(sem_id, &buf, 1);
    printf("释放资源完成\n");
    return 0;
}
```

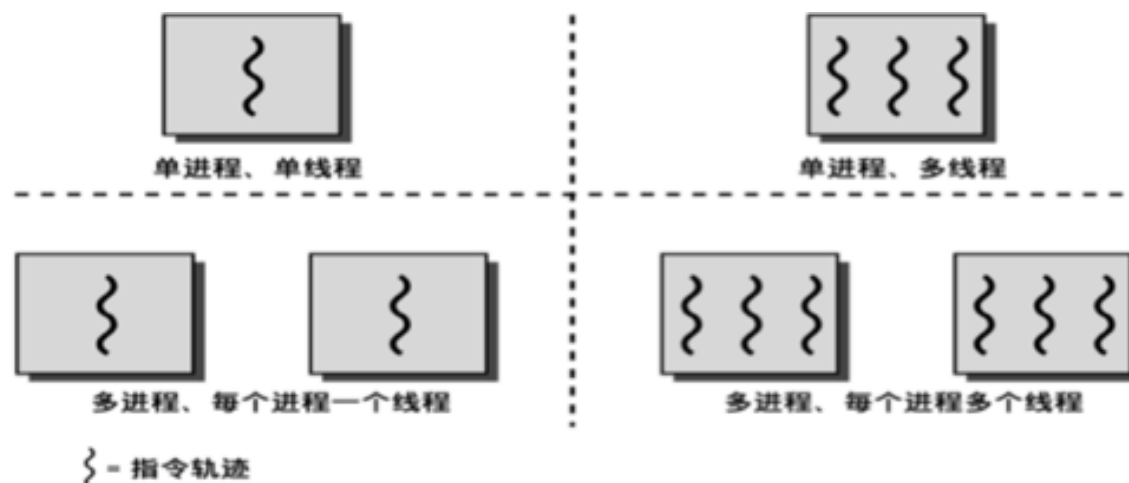
线程

进程和线程的区别

- (1) 计算机的核心是CPU，它承担了所有的计算任务。它就像一座工厂，时刻在运行。
- (2) 进程就好比工厂的车间，它代表CPU所能处理的单个任务。任一时刻，CPU总是运行一个进程，其他进程处于非运行状态。
- (3) 一个车间里，可以有很多工人。他们协同完成一个任务。
- (4) 线程就好比车间里的工人。多个线程为了各自完成属于自己的那份任务，从而使一个工厂能正常运行。一个进程可以包括多个线程。

线程的概念

- (1) 线程自己不拥有系统资源，它可与同属一个进程的其它线程共享进程所拥有的全部资源
- (2) 同一进程中的多个线程之间可以并发执行
- (3) 一个线程率属于创建它的进程
- (4) 一个进程必须要有一个主线程



线程属性

- (1) 线程ID
- (2) 程序计数器pc
- (3) 堆栈
- (4) 优先级
- (5) 执行的状态

线程ID

◆ 线程是存在于各个进程，所以线程ID是相对于当前进程里面其他线程的标志，不同进程里面的线程有可能相同

◆ 获取进程ID

函数原型： `pthread_t pthread_self(void);`

头文件： `#include<pthread.h>`

线程的创建

函数原型：`int pthread_create(pthread_t *restrict thread, const pthread_attr_t *restrict attr, void *(*start_routine)(void*), void *restrict arg)`

头文件：`#include <pthread.h>`

参数：参数一：存储线程的标志符的地址

参数二：线程的属性，不关心就给NULL

参数三：线程的入口地址，（函数名）

参数四：传递给线程函数的信息

返回值：成功为0，失败非0

等待线程的退出

◆思考为什么需要等待线程的结束?

函数原型: `int pthread_join(pthread_t, void * _Nullable * _Nullable);`

参数: 参数一: 等待哪个线程的结束, 线程的标识符

参数二: 存储等待的那个线程的退出码, 不关心为NULL

线程退出

函数原型：void pthread_exit(void * _Nullable);

参数：退出码

注意点：此处的退出码可以由pthread_join的第二个参数提取出来

练习

- ◆ 练习：往文件里面写“hello world”；要求子线程往文件里面写数据，主线程从文件里面读数据

线程同步和互斥

线程同步

◆1.什么是线程同步?

同步就是协同步调，按预定的先后次序进行运行。如：你说完，我再说

思考:怎么解决同步?

信号量

◆什么是信号量?

- (1) 信号量的使用主要是用来保护共享资源，使得资源在一个时刻只有一个进程（线程）所拥有
- (2) 信号量的值为正的时候，说明它空闲。所测试的线程可以锁定而使用它。
- (3) 信号量的值为0，说明它被占用，测试的线程/进程要进入睡眠队列中，等待被唤醒

◆信号量代表某一类资源，其值表示系统中该资源的数量

◆信号量是一个受保护的变量，只能通过三种操作来访问

- ◆初始化

- ◆ P 操作(申请资源)

- ◆ V 操作(释放资源)

◆信号量的值为非负整数

初始化信号量

函数原型： `int sem_init(sem_t *, int, unsigned int);`

原型： `#include <semaphore.h>`

参数： 参数一： 信号量的地址

参数二： 信号量的共享范围(0代表线程，1代表进程)

参数三： 信号量的资源值

信号量的操作

功能：申请资源

函数原型：int sem_wait(sem_t *);

参数：信号量的地址

注意：信号量资源减1

功能：释放资源

函数原型：int sem_post(sem_t *);

参数：信号量的地址

注意：信号量资源加1

功能：销毁信号量

函数原型：int sem_destroy(sem_t *);

参数：信号量的地址

线程互斥

◆什么是线程互斥?

某一资源同时只允许一个访问者对其进行访问，具有唯一性和排它性。但互斥无法限制访问者对资源的访问顺序，即访问是无序的

思考：怎么解决互斥?

- (1) 使用单个信号量
- (2) 使用互斥锁

互斥锁的操作

功能：创建互斥锁

函数原型：int pthread_mutex_init(pthread_mutex_t* __restrict, const pthread_mutexattr_t* __Nullable __restrict);

参数：参数一：互斥锁的地址

参数二：互斥锁的属性

功能：给当前线程加锁

函数原型：int pthread_mutex_lock(pthread_mutex_t*);

参数：互斥锁的地址

功能：给当前线程解锁

函数原型：int pthread_mutex_unlock(pthread_mutex_t*);

参数：互斥锁的地址

功能：毁掉锁

函数原型：int pthread_mutex_destroy(pthread_mutex_t*);

参数：互斥锁的地址

示例代码 1 of 4

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
```

```
int value1, value2;
pthread_mutex_t mutex;
unsigned int count = 0;
```

```
void *function(void *arg);
```

```
int main(int argc, char *argv[])
{
    pthread_t a_thread;
```

```
    if (pthread_create(&a_thread, NULL, function, NULL) < 0)
```

示例代码 2 of 4

```
{
    perror("fail to pthread_create");
    exit(-1);
}
if (pthread_mutex_init(&mutex, NULL) < 0)
{
    perror("fail to mutex_init");
    exit(-1);
}

while ( 1 )
{
    count++;
#ifdef _LOCK_
    pthread_mutex_lock(&mutex);
#endif
```


示例代码 3 of 4

```
        value1 = count;
        value2 = count;
#ifdef _LOCK_
        pthread_mutex_unlock(&mutex);
#endif
    }

    return 0;
}

void *function(void *arg)
{
    while ( 1 )
    {
#ifdef _LOCK_
        pthread_mutex_lock(&mutex);
#endif
```

示例代码 4 of 4

```
    if (value1 != value2)
    {
        printf("count=%d , value1=%d, value2=%d\n", count, value1,
value2);
    }
#ifdef _LOCK_
    pthread_mutex_unlock(&mutex);
#endif
}
return NULL;
}
```